

File Directory Management e Attacco Brute-Force ad un servizio SSH

03/06/2026

OBIETTIVO: *Completamento del gioco Gameshell.sh e Scrittura di un programma in Python che permetta l'esecuzione di un attacco Brute-Force ad un servizio SSH*

1. GAMESHELL

Introduzione

Immagina il sistema operativo del tuo computer (come Linux o Windows) non come una serie di icone colorate su cui cliccare, ma come una **grandissima fortezza organizzata** all'interno di questa fortezza ci sono stanze (**le cartelle**), documenti (**i file**), lavoratori che eseguono compiti (**i processi**) e guardiani che controllano chi può fare cosa (**i permessi**). Questa relazione è **la mappa e il manuale d'istruzioni** di quella fortezza, in cui, attraverso 44 livelli esploreremo come muoverci tra le stanze, come trovare documenti segreti o nascosti, come gestire i lavoratori (anche quando si ribellano o si comportano in modo strano) e come proteggere le informazioni da sguardi indiscreti

Suddivisione in Macro-gruppi

Per semplificare la comprensione della funzionalità dei livelli del gioco, si possono dividere in 4 macro-gruppi logici:

- 1. Macro-gruppo 1: **Navigazione ed Esplorazione del File System** (Livelli 1-12)
 - Focus: Orientamento nello spazio logico, gestione di file e cartelle, ricerca di elementi nascosti o basati su pattern specifici
- 2. Macro-gruppo 2: **Manipolazione, Filtraggio e Analisi dei Dati** (Livelli 13-25)
 - Focus: Ispezione del contenuto dei file, estrazione di stringhe, filtraggio di testo, reindirizzamento dei flussi di input/output e pipeline
- 3. Macro-gruppo 3: **Gestione dei Processi e Segnali di Sistema** (Livelli 26-35)
 - Focus: Monitoraggio dell'attività del sistema, cicli di vita dei processi, gestione dei job in background e interazione tramite segnali (es. INT, KILL, STOP, TERM)
- 4. Macro-gruppo 4: **Permessi, Privilegi e Amministrazione** (Livelli 36-45)
 - Focus: Controllo degli accessi (lettura, scrittura, esecuzione), utenti e gruppi, escalation di privilegi controllata e configurazioni di sicurezza

2. RELAZIONE TECNICA ED ANALITICA DEI LIVELLI

Macro-gruppo 1: Navigazione ed Esplorazione del File System (Livelli 1-12)

Questo blocco si concentra sull'interazione fondamentale con l'ambiente **CLI** (Command Line Interface) in cui il sistema operativo organizza i dati in un albero gerarchico e saperlo mappare è il primo passo per l'**incident response** e la **ricerca di artefatti**

Specifiche dei Livelli

```
[mission 1] $ ls
First_floor

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd First_floor/

[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Second_floor

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Second_floor/

[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Top_of_the_tower

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Top_of_the_tower

[use 'gsh help' to get a list of available commands]
[mission 1] $ gsh check

Congratulations, mission 1 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]
```

Livello 1 (Navigazione Lineare): Acquisizione della consapevolezza spaziale, il sistema richiede di scendere lungo una gerarchia di directory nidificate (First_floor/Second_floor/Top_of_the_tower) per localizzare un artefatto specifico

```
[use 'gsh help' to get a list of available commands]
[mission 2] $ cd ../../../../Castle/

[use 'gsh help' to get a list of available commands]
[mission 2] $ pwd
/home/kali/gameshell.1/World/Castle

[use 'gsh help' to get a list of available commands]
[mission 2] $ ls
Cellar  Great_hall  Main_building  Main_tower  Observatory

[use 'gsh help' to get a list of available commands]
[mission 2] $ cd Cellar/

[use 'gsh help' to get a list of available commands]
[mission 2] $ gsh check

Congratulations, mission 2 has been successfully completed!
```

Livello 2 (Spostamento Relativo Avanzato): Apprendimento dell'operatore di risalita “...” in cui l'utente impara a muoversi trasversalmente tra rami diversi del file system senza ripartire dalla radice, passando da una torre profonda alle cantine (Cellar)

```
[mission 3] $ cd

[use 'gsh help' to get a list of available commands]
[mission 3] $ cd Castle/Main_building/Throne_room

[use 'gsh help' to get a list of available commands]
[mission 3] $ gsh check

Congratulations, mission 3 has been successfully completed!
```

Livello 3 (Ritorno alla Home e Percorsi Assoluti): Utilizzo del comando di cambio directory senza argomenti per sfruttare il collegamento nativo alla cartella home dell'utente, seguito dall'accesso mirato alla Throne_room , in soli 2 comandi

```
~/Forest
[mission 4] $ mkdir Hut

~/Forest
[mission 4] $ ls
Hut

~/Forest
[mission 4] $ cd Hut

~/Forest/Hut
[mission 4] $ mkdir Chest

~/Forest/Hut
[mission 4] $ ls
Chest

~/Forest/Hut
[mission 4] $ gsh check

Congratulations, mission 4 has been successfully completed!
```

Livello 4 (Inizializzazione di Strutture): Apprendimento della creazione di nodi logici, il sistema richiede la generazione di una directory (Hut) e di una sotto-directory nidificata (Chest) per strutturare uno spazio di archiviazione pulito

```
~/Castle/Cellar
[mission 5] $ ls
barrel_of_apples  bat_1  bat_2  spider_1  spider_2  spider_3

~/Castle/Cellar
[mission 5] $ rm spider_1 spider_2 spider_3

~/Castle/Cellar
[mission 5] $ ls
barrel_of_apples  bat_1  bat_2

~/Castle/Cellar
[mission 5] $ gsh check

Congratulations, mission 5 has been successfully completed!
```

Livello 5 (Sanificazione Manuale): Primo contatto con la rimozione mirata di risorse, si procede con l’eliminazione selettiva di file specifici (spider_1, spider_2, spider_3) all'interno di una stanza, preservando l'integrità degli altri elementi circostanti

```
~/Garden
[mission 6] $ ls
coin_1  coin_2  coin_3  Flower_garden  Maze  Shed

~/Garden
[mission 6] $ mv coin_1 coin_2 coin_3 ../Forest/Hut/Chest

~/Garden
[mission 6] $ cd ../Forest/Hut/Chest

~/Forest/Hut/Chest
[mission 6] $ ls
coin_1  coin_2  coin_3

~/Forest/Hut/Chest
[mission 6] $ gsh check

Congratulations, mission 6 has been successfully completed!
```

Livello 6 (Logistica e Riposizionamento): Comprensione dello spostamento dei file. Trasferimento fisico di una serie di risorse d'oro (coin_1, coin_2, coin_3) da una locazione remota (Garden) al contenitore protetto creato nel livello 4


```
~/Garden
[mission 7] $ ls -A
.33139_coin_2 .5742_coin_3 .64677_coin_1 Flower_garden Maze Shed

~/Garden
[mission 7] $ mv .33139_coin_2 .5742_coin_3 .64677_coin_1 ../Forest/Hut/Chest

~/Garden
[mission 7] $ gsh check

Congratulations, mission 7 has been successfully completed!
```

```
~/Castle/Cellar
[mission 8] $ ls
10539_spider_48 20754_spider_45 24620_spider_41 29430_spider_32 6062_spider_35
11221_spider_10 21015_spider_44 25513_spider_27 29483_spider_46 7716_spider_22
11276_spider_7 21294_spider_49 265_spider_12 29932_spider_34 8059_spider_37
12229_spider_24 21513_spider_25 26628_spider_47 31340_spider_4 8077_spider_29
1257_spider_36 21647_spider_2 26755_spider_17 31360_spider_50 9461_spider_33
14692_spider_42 21756_spider_8 26789_spider_3 3204_spider_28 9472_spider_40
16958_spider_18 22119_bat_3 26953_spider_15 32698_spider_43 9862_spider_23
17772_spider_31 22462_spider_26 28378_spider_5 4195_bat_4 barrel_of_apples
18915_bat_1 22506_bat_2 28760_spider_13 5273_spider_14
19339_spider_1 2274_spider_38 29244_spider_30 5581_spider_39
20027_spider_16 22932_spider_19 29253_spider_9 5791_spider_11
202_spider_20 23305_bat_5 29322_spider_6 5997_spider_21

~/Castle/Cellar
[mission 8] $ rm *spi*

~/Castle/Cellar
[mission 8] $ ls
18915_bat_1 22119_bat_3 22506_bat_2 23305_bat_5 4195_bat_4 barrel_of_apples

~/Castle/Cellar
[mission 8] $ gsh check

Congratulations, mission 8 has been successfully completed!
```

```
[mission 9] $ ls -a
. .17411_spider_10 .24952_bat_3 4195_bat_4
.. .18718_spider_31 .25511_spider_9 .4346_spider_35
.10070_spider_29 18915_bat_1 .25822_spider_30 .4642_spider_43
.10197_spider_40 .19590_spider_36 .26724_spider_5 .4671_bat_1
.10766_spider_15 .19593_spider_34 .27205_spider_50 .544_spider_3
.1097_bat_2 .20788_spider_11 .28131_spider_20 .5805_spider_38
.11000_spider_48 .21213_spider_18 .28390_spider_49 .651_spider_13
.1115_spider_23 .21450_spider_25 .28846_spider_24 .6570_spider_44
.11967_spider_32 .21519_spider_6 .30087_spider_39 .6683_spider_2
.12352_spider_37 .21902_spider_26 .30273_spider_21 .7403_spider_22
.13466_spider_41 22119_bat_3 .31260_spider_16 .851_spider_1
.14157_spider_7 22506_bat_2 .31262_spider_12 .8811_spider_33
.1571_bat_5 .22556_spider_46 .32303_spider_17 .9396_spider_47
.15778_bat_4 .22720_spider_8 .32327_spider_4 .9755_spider_27
.15844_spider_42 23305_bat_5 .32500_spider_28 barrel_of_apples
.16148_spider_19 .24346_spider_45 .372_spider_14

~/Castle/Cellar
[mission 9] $ rm .*spi*

~/Castle/Cellar
[mission 9] $ ls -a
. .1571_bat_5 22119_bat_3 .24952_bat_3 barrel_of_apples
.. .15778_bat_4 22506_bat_2 4195_bat_4
.1097_bat_2 18915_bat_1 23305_bat_5 .4671_bat_1

~/Castle/Cellar
[mission 9] $ gsh check

Congratulations, mission 9 has been successfully completed!
```

```
~/Castle/Great_hall
[mission 10] $ ls
22556_decorative_shield 46042_stag_head standard_2 standard_4
30383_suit_of_armour standard_1 standard_3

~/Castle/Great_hall
[mission 10] $ cp standard_1 standard_2 standard_3 standard_4 ../Forest/Hut/Chest

~/Castle/Great_hall
[mission 10] $ ls
22556_decorative_shield 46042_stag_head standard_2 standard_4
30383_suit_of_armour standard_1 standard_3

~/Castle/Great_hall
[mission 10] $ cd ../Forest/Hut/Chest

~/Forest/Hut/Chest
[mission 10] $ ls
coin_1 coin_2 coin_3 standard_1 standard_2 standard_3 standard_4

~/Forest/Hut/Chest
[mission 10] $ gsh check

Congratulations, mission 10 has been successfully completed!
```

```
[mission 11] $ ls
23670_tapestry_06 38738_tapestry_03 57932_stag_head standard_3
31941_tapestry_09 39170_tapestry_10 62534_tapestry_07 standard_4
32716_decorative_shield 43084_tapestry_01 63643_tapestry_05
33686_tapestry_02 47335_tapestry_08 standard_1
34755_suit_of_armour 50110_tapestry_04 standard_2

~/Castle/Great_hall
[mission 11] $ cp *tap* ../Forest/Hut/Chest
cp: target '../Forest/Hut/Chest': No such file or directory

~/Castle/Great_hall
[mission 11] $ cp *tap* ../../Forest/Hut/Chest

~/Castle/Great_hall
[mission 11] $ gsh check

Congratulations, mission 11 has been successfully completed!
```

```
~/Castle/Main_tower/First_floor
[mission 12] $ ls -l
total 16
-rw-rw-r-- 1 kali kali 1501 Sep 19 1986 painting_ASVywXXs
-rw-rw-r-- 1 kali kali 1055 Mar 3 2011 painting_SAogKHem
-rw-rw-r-- 1 kali kali 1455 Nov 19 2003 painting_VDIwNdKZ
drwxrwxr-x 3 kali kali 4096 May 30 07:19 Second_floor/

~/Castle/Main_tower/First_floor
[mission 12] $ cp painting_ASVywXXs ../../Forest/Hut/Chest

~/Castle/Main_tower/First_floor
[mission 12] $ gsh check

Congratulations, mission 12 has been successfully completed!
```

Livello 7 (Offuscamento tramite Attributo Nascosto): Studio delle convenzioni Unix sui file nascosti, in cui si effettuerà l’identificazione di un file il cui nome comincia con un punto (.) e successiva normalizzazione tramite spostamento in un'area visibile

Livello 8 (Pattern Matching - Espansione dei Caratteri Jolly): Introduzione all'efficienza di amministrazione, con l’uso del carattere speciale “*” per eliminare massivamente decine di file contenenti la stringa "spider" in un unico ciclo di clock, evitando così la selezione manuale

Livello 9 (Pattern Matching Avanzato e File Nascosti): Fusione delle due competenze precedenti, quindi la pulizia di una stanza mediante l'eliminazione mirata di file che sono sia nascosti sia rispondenti a un determinato pattern di testo (*.spi*)

Livello 10 (Integrità dei Dati e Duplicazione): Studio della copia delle risorse, invece di spostare il file originale, viene creata una copia esatta di un artefatto per scopi di backup o analisi all'interno del forziere di destinazione (cp)

Livello 11 (Creazione di Collegamenti Simbolici): Concetto di puntatore nel file system, c’è la generazione di un link simbolico (symlink) che funge da scorciatoia logica per connettere un'area distante del sistema a una cartella di lavoro rapido

Livello 12 (Ispezione delle Proprietà Geometriche): Utilizzo di strumenti di scansione interna per individuare risorse non in base al nome, ma in base alle loro proprietà strutturali fisiche (es. dimensione esatta in byte), per poi prendere il quadro più antico e spostarlo nella directory creata

Comando	Sintassi Comune	Funzione Tecnica
<i>pwd</i>	pwd	Mostra il percorso assoluto della directory di lavoro corrente
<i>ls</i>	ls -la [cartella]	Elenca i contenuti; -l mostra i dettagli estesi, -a include i file nascosti
<i>cd</i>	cd /percorso/o/..	<i>Cambia la directory corrente; cd senza argomenti riporta alla Home utente</i>
<i>mkdir</i>	<i>mkdir -p nome_cartella</i>	Crea una directory; -p genera in automatico le cartelle intermedie se mancanti
<i>cp</i>	cp -r sorgente destinazione	Duplica file; -r abilita la copia ricorsiva delle intere strutture di directory
<i>mv</i>	mv sorgente destinazione	<i>Sposta una risorsa in una nuova cartella</i>
<i>rm</i>	rm -rf nome_file	Elimina risorse; -r ricorsivo per cartelle, -f forza l'azione senza chiedere conferme
<i>ln</i>	ln -s target link_name	Genera un collegamento simbolico (-s) che punta alla risorsa target

Macro-gruppo 2: Manipolazione, Filtraggio e Analisi dei Dati (Livelli 13-25)

L'analisi dei dati e dei file di log è l'attività primaria per comprendere lo stato di salute di un sistema o rilevare intrusioni e questo blocco insegna a estrarre informazioni utili da flussi di testo massivi

Specifiche dei Livelli

```
~/Castle/Main_tower/First_floor
[mission 13] $ cal 09 1902
      September 1902
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30

~/Castle/Main_tower/First_floor
[mission 13] $ gsh check
What was the day of the week for the 09-20-1902?
 1 : Monday
 2 : Tuesday
 3 : Wednesday
 4 : Thursday
 5 : Friday
 6 : Saturday
 7 : Sunday
Your answer: 6

Congratulations, mission 13 has been successfully completed!
```

Livello 13 (Calendario con *cal*): Mediante “*cal 09 1902*” si visualizza il calendario di settembre 1902, dal quale si determina il giorno della settimana corrispondente alla data 20/09/1902. ossia sabato

```
~/Castle/Main_tower/First_floor
[mission 14] $ alias la='ls -A'

~/Castle/Main_tower/First_floor
[mission 14] $ la
.nice_rock  painting_ASVywXXs  painting_SAogKHem  painting_VDIwNdKZ  Second_floor/

~/Castle/Main_tower/First_floor
[mission 14] $ gsh check

Congratulations, mission 14 has been successfully completed!
```

Livello 14 (*Alias* e file nascosti): Si definisce l’alias “*la=’ls-A’*” e lo si impiega per elencare il contenuto della directory, inclusi i file nascosti

```
[mission 15] $ nano journal.txt

~
[mission 15] $ gsh check

Congratulations, mission 15 has been successfully completed!
```

Livello 15 (Editing con *nano*): Si crea ed dita il file *journal.txt* tramite l’editor di testo *nano*

```
~/Forest/Hut/Chest
[mission 16] $ alias journal='nano ~/Forest/Hut/Chest/journal.txt'

~/Forest/Hut/Chest
[mission 16] $ gsh check

Congratulations, mission 16 has been successfully completed!
```

Livello 16 (*Alias* per apertura file): si definisce l’alias *journal=’nano ~/Forest/Hut/Chest/journal.txt* , che consente l’apertura diretta del file in *nano*

```
~/Castle/Cellar
[mission 17] $ cd .Lair_of_the_spider_queen\ MzUThCcvckZoopIh pvlgBzq0hqtFYIay/

~/Castle/Cellar/.Lair_of_the_spider_queen MzUThCcvckZoopIh pvlgBzq0hqtFYIay
[mission 17] $ la
IGcOrwxBgvyXZWcK_spider_queen_fxCBzkAKPizkKiZZ
LXXyXqCgngOLrojS_baby_bat_cPJVa0sqyvMsnlKJ

~/Castle/Cellar/.Lair_of_the_spider_queen MzUThCcvckZoopIh pvlgBzq0hqtFYIay
[mission 17] $ rm IGcOrwxBgvyXZWcK_spider_queen_fxCBzkAKPizkKiZZ

~/Castle/Cellar/.Lair_of_the_spider_queen MzUThCcvckZoopIh pvlgBzq0hqtFYIay
[mission 17] $ gsh check
Perfect, it took you only 16 seconds to complete this mission!
```

Livello 17 (*Path* con caratteri speciali a tempo): Si accede, mediante *cd*, a una directory il cui nome contiene spazi e caratteri speciali (lo spazio è gestito tramite escape con `\`); si elenca quindi il contenuto con l'alias *la* e si elimina con *rm* il file relativo alla spider queen


```
~/Castle
[mission 18] $ xeyes &
[1] 211266

~/Castle
[mission 18] $ xeyes
c^C

~/Castle
[mission 18] $ ^C

~/Castle
[mission 18] $ gsh check

Congratulations, mission 18 has been successfully completed!
```

```
~/Castle
[mission 19] $ flarigo & flarigo & flarigo & gsh check
[2] 217556
[3] 217557
[4] 217558
```

```
Great, that looked good!
[2] Done flarigo
[3]- Done flarigo
[4]+ Done flarigo

Congratulations, mission 19 has been successfully completed!
```

```
~/Castle
[mission 20] $ gsh check
What's a valid 4 letters sequence? zblp

Congratulations, mission 20 has been successfully completed!
```

```
[mission 20] $ charmiglio zblp
#.-:~%:_.*::#-:~.*
( )_.(*.*.))#.%-))
)*.%(.*.)(-)*.(..
.*.))%#)%:#..)_)*
.-%: ... ** ::-))!:#
:~%: .. *#_*_.(#%):.
%:#.#):(-)*.*%)-
)(%(.##*!):.-:- ... -*
:-_:#*!:_-:~%*..*~*%
.~%:(.:-_#(##)
.%(._._.#)!.*)
.._(_)_#:#:~(-)(.~.%%
-_-#(-))_-#~#%_
((._.%_#)##.)(.:-
%~:-:(###-~%#.:#:
#:#(_))((%*::_-:~(-
~%_~%~:~%.(.(-)
%:(.*%:_._.*%.-~%
```

It works! The special incantation is zblp

```
~/Castle
[mission 21] $ find ~/ -name "*copper*" -exec mv {} ~/Forest/Hut/Chest/ \;
mv: '/home/kali/gameshell.1/World/Forest/Hut/Chest/00000_copper_coin_00000' and '/home/k
ali/gameshell.1/World/Forest/Hut/Chest/00000_copper_coin_00000' are the same file

~/Castle
[mission 21] $ gsh check

Congratulations, mission 21 has been successfully completed!
```

```
[mission 22] $ find ~/ -name "*silver*" -exec mv {} ~/Forest/Hut/Chest/ \;
mv: '/home/kali/gameshell.1/World/Forest/Hut/Chest/00000_silver_coin_00000' and '/home/k
ali/gameshell.1/World/Forest/Hut/Chest/00000_silver_coin_00000' are the same file

~
[mission 22] $ gsh check

Congratulations, mission 22 has been successfully completed!
```

```
~/Garden/Maze
[mission 23] $ gsh check

Congratulations, mission 23 has been successfully completed!
```

```
~/Mountain/Cave
[mission 24] $ head -n 6 Book_of_potions/page_07
vvvvvvvvvv
Herbal tea
^^^^^^^^^^

1) Boil water.
2) Add herbs from the forest.
3) Let it sit for five minutes and drink while hot.

~/Mountain/Cave
[mission 24] $ gsh check

Congratulations, mission 24 has been successfully completed!
```

```
~/Mountain/Cave
[mission 25] $ tail -n 9 Book_of_potions/page_12
1) Boil water in a cauldron.
2) Add in a few death caps (Amanita phalloides).
3) Also add a few fly agarics (Amanita muscaria).
4) And some destroying angels (Amanita virosa).
5) Mix in a few deadly webcaps (Cortinarius rubellus).
6) Feel free to add in any colourful fungi you have on hand.
7) Let half of the water evaporate.
8) Season with a pinch of salt and a few herbs.
9) Serve hot in a bowl.

~/Mountain/Cave
[mission 25] $ gsh check

Congratulations, mission 25 has been successfully completed!
```

Mission 18 (Processo in background e interruzione) — Si avvia il processo *xeyes &* in background e se ne gestisce la terminazione tramite l'interruzione da tastiera Ctrl-C (^C)

Mission 19 (Processi paralleli in background) — Si avviano contestualmente tre istanze del processo *flarigo* in background mediante l'operatore *&* , la cui esecuzione si conclude regolarmente con stato *"Done"*

Mission 20 (Comando personalizzato e validazione) — La missione richiede l'individuazione della sequenza valida di quattro lettere e l'esecuzione del comando *charmiglio zblp* restituisce l'incantesimo speciale e ne conferma il valore *zblp*, successivamente fornito in fase di validazione tramite *gsh check*

Mission 21 (Ricerca ed esecuzione con find -exec) — Si procede alla localizzazione e al trasferimento della moneta di rame mediante *find ~/ -name "*copper*" -exec mv {} ~/Forest/Hut/Chest/ \;*

Mission 22 (find -exec: moneta d'argento) — La missione replica la logica precedente applicando il costrutto *find ... -exec mv* alla moneta d'argento, trasferita nel forziere *~/Forest/Hut/Chest/*

Mission 23 (Find con negazione e move) — Trovare la vera moneta d'oro tra *"GoId_CoiN_2"* e *"gold_coin_1"* (case-sensitive!). Usare *'find'* con *'! -name'* per escludere *gold_coin_1* e spostare il resto nel *Chest* , la vera moneta non si chiama *"gold_coin_!"* (maiuscole diverse)

Mission 24 (Lettura iniziale con head) — Si effettua la lettura delle prime sei righe della pagina mediante *head -n 6 Book_of_potions/page_07*, ottenendo la porzione iniziale del documento (ricetta del tè alle erbe)

Mission 25 (Lettura finale con tail) — In modo complementare alla precedente, si estraggono le ultime nove righe della pagina con *tail -n 9 Book_of_potions/page_12*


```
ps
  PID TTY          TIME CMD
 2231 pts/0    00:00:00 zsh
 2475 pts/0    00:00:00 bash
 2545 pts/0    00:00:00 bash
 3399 pts/0    00:00:00 spell
 4009 pts/0    00:00:00 ps

~
[mission 29] $

      *#@*
      6_**/~
      !$-#

k      *#@*
      6_**/~
      !$-#

      *#@*
      6_**/~
      !$-#

kill 3399

~

~
[mission 29] $ gsh check

Congratulations, mission 29 has been successfully completed!
```

Mission 29 (Terminazione di processo con segnale predefinito) — Si individua con *ps* il *PID del processo spell (3399)* e lo si termina con **kill 3399**

```
ps
  PID TTY          TIME CMD
 28572 pts/0    00:00:01 zsh
 29011 pts/0    00:00:00 bash
 29088 pts/0    00:00:00 bash
 29866 pts/0    00:00:00 spell
 30184 pts/0    00:00:00 ps

~
[mission 30] $

      *#@*
      6_**/~
      !$-#

kill -s KILL 29
      *#@*
      6_**/~
      !$-#

866

~
[mission 30] $ gsh check

Congratulations, mission 30 has been successfully completed
```

Mission 30 (Terminazione forzata di processo) — Si individua con *ps* il *PID del processo spell (29866)* e lo si termina con segnale *KILL* mediante **kill -s KILL 29866**

```
~/Castle/Cellar
[mission 31] $ pstree -pa 291427
bash,291427
├── mischievous_imp,837504 /home/kali/gameshell.3/.tmp/mischievous_imp
│   └── tail,837528 -f /dev/null
├── nice_fairy,837502 /home/kali/gameshell.3/.tmp/nice_fairy
│   ├── spell,837509 /home/kali/gameshell.3/.tmp/fairy/spell 0
│   │   └── sleep,907420 3
│   ├── spell,837510 /home/kali/gameshell.3/.tmp/fairy/spell 1
│   │   └── sleep,907465 3
│   ├── spell,837511 /home/kali/gameshell.3/.tmp/fairy/spell 2
│   │   └── sleep,907517 3
│   └── tail,837512 -f /dev/null
└── pstree,907518 -pa 291427

~/Castle/Cellar
[mission 31] $ gsh check

Congratulations, mission 31 has been successfully completed!
```

Mission 31 (Ispezione dell'albero dei processi) — Con *pstree -pa 291427* si visualizza l'albero dei processi (con PID e argomenti), individuando *mischievous_imp* e *nice_fairy* con i rispettivi processi figli *spell*

```
~/Castle/Cellar
[mission 32] $ gsh check
30 + 98 = ?? 128
78 + 96 = ?? 174
44 + 49 = ?? 93
50 + 40 = ?? 90
63 + 34 = ?? 97

Congratulations, mission 32 has been successfully completed!
```

Mission 32 (Validazione con operazioni aritmetiche) — La validazione *gsh check* propone una sequenza di addizioni, a ciascuna delle quali viene fornito il risultato numerico corrispondente

```
~/Castle/Main_building/Library
[mission 33] $ gsh check < Mathematics_101
92 * 22 = ?? 16 * 68 = ?? 92 * 22 = ?? 19 * 43 = ?? 20 * 51 = ?? 41 * 61 = ?? 51 * 7 = ?
? 48 * 4 = ?? 84 * 22 = ?? 37 * 7 = ?? 91 * 64 = ?? 15 * 56 = ?? 27 * 56 = ?? 24 * 81 =
?? 71 * 94 = ?? 44 * 29 = ?? 91 * 49 = ?? 66 * 78 = ?? 66 * 56 = ?? 71 * 56 = ?? 4 * 1 =
?? 8 * 14 = ?? 40 * 32 = ?? 97 * 86 = ?? 76 * 50 = ?? 5 * 95 = ?? 48 * 89 = ?? 85 * 43
= ?? 41 * 16 = ?? 29 * 1 = ?? 71 * 63 = ?? 55 * 80 = ?? 92 * 94 = ?? 1 * 2 = ?? 3 * 68 =
?? 55 * 84 = ?? 57 * 27 = ?? 95 * 86 = ?? 13 * 82 = ?? 46 * 7 = ?? 70 * 92 = ?? 18 * 54
= ?? 70 * 88 = ?? 90 * 31 = ?? 15 * 30 = ?? 69 * 67 = ?? 58 * 63 = ?? 94 * 91 = ?? 4 *
5 = ?? 90 * 36 = ?? 83 * 38 = ?? 63 * 92 = ?? 16 * 33 = ?? 16 * 2 = ?? 73 * 72 = ?? 42 *
92 = ?? 97 * 47 = ?? 27 * 65 = ?? 35 * 38 = ?? 95 * 32 = ?? 76 * 44 = ?? 11 * 73 = ?? 2
4 * 30 = ?? 3 * 82 = ?? 88 * 67 = ?? 93 * 58 = ?? 73 * 75 = ?? 39 * 62 = ?? 99 * 61 = ??
57 * 71 = ?? 71 * 63 = ?? 23 * 12 = ?? 99 * 20 = ?? 90 * 88 = ?? 35 * 9 = ?? 44 * 42 =
?? 6 * 83 = ?? 56 * 47 = ?? 98 * 20 = ?? 60 * 24 = ?? 23 * 38 = ?? 22 * 98 = ?? 19 * 99
= ?? 3 * 79 = ?? 79 * 15 = ?? 64 * 42 = ?? 47 * 60 = ?? 93 * 7 = ?? 38 * 1 = ?? 99 * 78
= ?? 46 * 27 = ?? 16 * 55 = ?? 6 * 3 = ?? 25 * 33 = ?? 6 * 81 = ?? 8 * 41 = ?? 44 * 31 =
?? 33 * 55 = ?? 29 * 8 = ?? 96 * 96 = ??

Congratulations, mission 33 has been successfully completed!
```

Mission 33 (Redirezione dell'input alla validazione) — Si reindirizza il file *Mathematics_101* come standard input di *gsh check (gsh check < Mathematics_101)*, fornendo automaticamente le risposte alla serie estesa di moltiplicazioni a tempo

```
~/Castle/Main_building/Library/Merlin_s_office
[mission 34] $ cd Drawer

~/Castle/Main_building/Library/Merlin_s_office/Drawer
[mission 34] $ ls
ink_and_scroll  inventory.txt

~/Castle/Main_building/Library/Merlin_s_office/Drawer
[mission 34] $ less inventory.txt

~/Castle/Main_building/Library/Merlin_s_office/Drawer
[mission 34] $ gsh check

Congratulations, mission 34 has been successfully completed!
```

Mission 34 (Navigazione e consultazione con pager) — Si accede alla directory *Drawer* con *cd*, se ne elenca il contenuto con *ls* e si consulta il file *inventory.txt* tramite il pager less

```
~/Castle/Main_building/Library/Merlin_s_office
[mission 35] $ gsh check

Congratulations, mission 35 has been successfully completed!
```

Mission 35 (Permessi e Accesso Directory) — con il comando *chmod +x(esecuzione) King_s_quarter* diamo il permesso ad accedere alla stanza, qua vediamo solo il check finale

Comando	Sintassi Comune	Funzione Tecnica
<i>ps</i>	ps	Elenca i processi attivi nella sessione con PID, terminale, tempo CPU e nome del comando
<i>kill</i>	<i>kill PID</i>	Invia un segnale a un processo
<i>KILL</i>	kill -s KILL PID	<i>Terminazione forzata e immediata non intercettabile dal processo</i>
<i>pstree</i>	<i>pstree -pa PID</i>	Mostra la gerarchia dei precessi -p (aggiunge i PID) -a (aggiunge gli argomenti di riga di comando)
<i>less</i>	less FILE	Pager per consultazione interattiva e paginata di un file, senza caricarlo interamente in memoria
<i>pipe</i>	comando1 comando2	<i>Reindirizza lo standard output del primo comando allo standard input del secondo</i>

Macro-gruppo 4: Permessi, Identità e Amministrazione Privilegiata (Livelli 36-44)

Questo blocco finale esplora come vengono protetti i file e come si gestiscono le identità e l'escalation dei privilegi

Specifiche dei livelli

```
~/Castle/Observatory
[mission 36] $ gsh check < err
What is the secret key?
Congratulations, mission 36 has been successfully completed!
```

Mission 36 (Redirezione dell'input alla validazione) — Si reindirizza il file `err` come standard input di ***gsh check*** (***gsh check < err***), fornendo la chiave segreta richiesta

```
~/Castle/Main_building/Throne_room
[mission 37] $ chmod +x Kings_quarter/

~/Castle/Main_building/Throne_room
[mission 37] $ cd Kings_quarter/

~/Castle/Main_building/Throne_room/Kings_quarter
[mission 37] $ gsh check

Congratulations, mission 37 has been successfully completed!
```

Mission 37 (Permesso di attraversamento di directory) — Si attribuisce il permesso di esecuzione alla directory con `chmod +x Kings_quarter/`, condizione necessaria per accedervi con ***cd***

```
~/Castle/Main_building/Throne_room/Kings_quarter
[mission 38] $ cat .secret_note
1563660921

~/Castle/Main_building/Throne_room/Kings_quarter
[mission 38] $ gsh check
What's the combination to open the King's safe? 1563660921

Congratulations, mission 38 has been successfully completed!
```

Mission 38 (Lettura di file nascosto) — Si legge il file nascosto ***.secret_note*** con ***cat***, ottenendo la combinazione della cassaforte (**1563660921**), poi fornita in validazione

```
~/Castle/Main_building/Throne_room/Safe
[mission 39] $ ls -l
total 4
----- 1 kali kali 48 May 31 14:18 crown

~/Castle/Main_building/Throne_room/Safe
[mission 39] $ chmod +r crown

~/Castle/Main_building/Throne_room/Safe
[mission 39] $ cat crown
jgs
(^~^~^~^~)
\@*@\*@/
{247_}

~/Castle/Main_building/Throne_room/Safe
[mission 39] $ mv crown ../../../../Forest/Hut/Chest

~/Castle/Main_building/Throne_room/Safe
[mission 39] $ ls

~/Castle/Main_building/Throne_room/Safe
[mission 39] $ gsh check
What are the 3 digits inscribed on the base of the crown? 247

Congratulations, mission 39 has been successfully completed!
```

Mission 39 (Gestione dei permessi di lettura) — Si rileva con ***ls -l*** che il file `crown` è privo di permessi; si abilita la lettura con `chmod +r crown`, se ne visualizza il contenuto con ***cat*** (le tre cifre alla base sono **247**) e lo si sposta nel ***Chest*** con ***mv***

```
~/Garden/Maze
[mission 40] $ grep -rli ruby .
./9bca7aab0f2df7746e/2356ca1b5ad/70a3003bce8ce/44696

~/Garden/Maze
[mission 40] $ mv ./9bca7aab0f2df7746e/2356ca1b5ad/70a3003bce8ce/44696 ~/Forest/Hut/Chest

~/Garden/Maze
[mission 40] $ gsh check

Congratulations, mission 40 has been successfully completed!
```

Mission 40 (Ricerca ricorsiva con grep) — Con ***grep -rli ruby*** . si cerca ricorsivamente, in modo case-insensitive e restituendo solo i nomi file, il file contenente "***ruby***", che viene poi spostato nel ***Chest*** con ***mv***


```
~/Garden/Maze
[mission 41] $ find . -type f | xargs grep -l diamond
./fb7f5baec/847d12aead6b/d13d5d2a

~/Garden/Maze
[mission 41] $ mv ./fb
fb7f5baec/          fbc3b63a3452881/

~/Garden/Maze
[mission 41] $ mv ./fb7f5baec/847d12aead6b/d13d5d2a ~/Forest/Hut/Chest

~/Garden/Maze
[mission 41] $ gsh check

Congratulations, mission 41 has been successfully completed!
```

```
(0)
~/Stall
[mission 42] $ cat * | grep King | grep -v PAID
the King bought a blanket for 3 coppers.
the King bought a leather bag for 5 coppers.
the King bought a wooden spoon for 2 coppers.
the King bought a blanket for 5 coppers.
the King bought a walking stick for 3 coppers.
(1)
~/Stall
[mission 42] $ gsh check
How much does the king owe? 18

Congratulations, mission 42 has been successfully completed!
```

```
~/Stall
[mission 43] $ gsh check
How many unpaid items are there? 54

Congratulations, mission 43 has been successfully completed!
```

```
~/Castle/Main_building/Library/Merlin_s_office/Drawer
[mission 44] $ tr 'a-zA-Z' 'o-za-nO-ZA-N' < secret_message
here is my will:
you will get my chest, and everything it contains.
this chest is in the cellar, and the word to make
it re-appear is: fana
merlin the enchanter

~/Castle/Main_building/Library/Merlin_s_office/Drawer
[mission 44] $ gsh check
What's the key that will make Merlin's chest to appear?
fana

Congratulations, mission 44 has been successfully completed!
```

Mission 41 (Ricerca per contenuto con find e xargs) — Si individua il file contenente "***diamond***" mediante ***find . -type f | xargs grep -l diamond*** e lo si trasferisce nel ***Chest*** con ***mv***

Mission 42 (Filtri concatenati e somma dei debiti) — Con ***cat * | grep King | grep -v PAID*** si concatenano i file e si isolano le righe relative al Re escludendo quelle saldate ("PAID"); la somma dei debiti residui (18) è fornita in validazione

Mission 43 (Conteggio degli articoli non saldati) — con ***cat* | grep -v PAID*** si isoleranno i conti non saldati totali, la validazione ***gsh check*** richiede il numero di articoli non saldati (54);

Mission 44 (Decifratura con cifrario a sostituzione) — dopo vari tentativi abbiamo ottenuto lo shift giusto (12) e si decifra il file ***secret_message*** con ***tr 'a-zA-Z' 'o-za-nO-ZA-N' < secret_message*** (cifrario a rotazione), ottenendo la chiave fana, poi fornita in validazione

Comando	Sintassi Comune	Funzione Tecnica
<i>grep</i>	grep parola FILE	Ricerca righe che corrispondono ad un pattern all’interno di uno o più file
<i>xargs</i>	... xargas comando	Converte un flusso di testo (da pipe) negli argomenti del comando successivo
<i>tr</i>	tr set1 set2 < FILE	<i>Traduce/sostituisce i caratteri di set1 nei caratteri di set2 (in questo caso usato come cifrario a rotazione)</i>
<i>chmod +x</i>	<i>chmod +x TARGET</i>	<i>Aggiunge il permesso di esecuzione, abilita l’attraversamneto di una directory</i>
<i>chmod +r</i>	<i>chmod +r FILE</i>	Aggiunge il permesso di lettura, condizione necessaria per consultarne il contenuto
<i>ls -l</i>	le -l	<i>Elenco in formato esteso: permessi, proprietario, dimensione e data di ciascun elemento</i>

3. ATTACCO BRUTE FORCE AD UN SERVIZIO SSH

Executive Summary

Nel corso di questa attività di verifica di sicurezza in ambiente isolato, è stato simulato ed eseguito un **attacco di tipo Brute Force** basato su dizionario contro il **servizio SSH di un host target locale**, l'attacco è stato automatizzato tramite uno script Python personalizzato basato sulla libreria **paramiko**, la relazione che segue mappa l'intero ciclo di vita del test, unendo l'analisi concettuale alle specifiche tecniche e ai comandi rilevati direttamente sulle macchine

Fase 1: Network Reconnaissance e Ispezione Directory

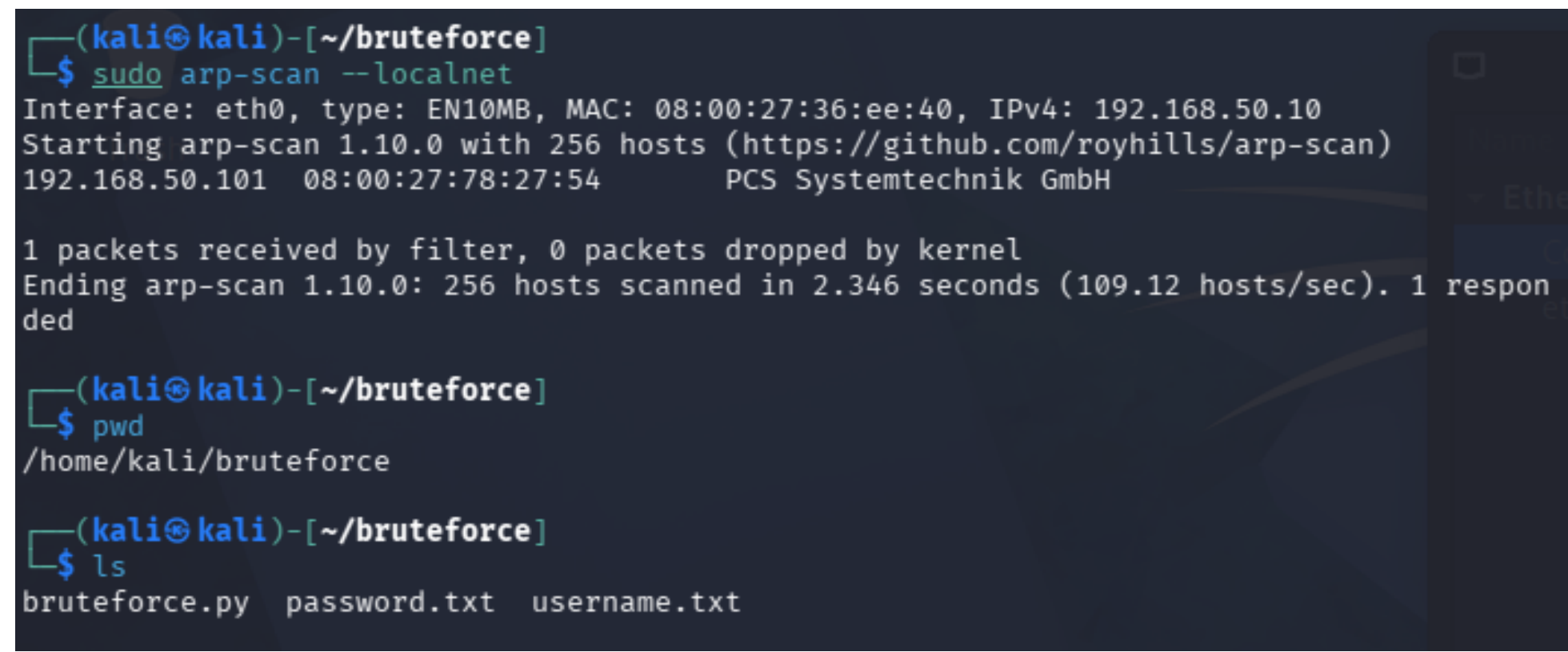


Fig. 1 — [individuazione del target e chiamata dei dizionari precedentemente creati]

Il test ha inizio con l'identificazione dei sistemi attivi nella subnet locale per individuare il target e la successiva verifica dello spazio di lavoro sulla macchina attaccante Kali Linux. Come documentato nella Figura 1 l'operatore esegue una **scansione ARP** scoprendo l'**host target con IP 192.168.50.101**. , subito dopo si verifica la cartella di lavoro /home/kali/bruteforce e accerta la presenza dei file necessari: **lo script** (bruteforce.py) e i **due dizionari** (password.txt e username.txt)

Comando	Flag/Argomenti	Funzione Tecnica
<i>sudo arp-scan --localnet</i>	<i>--localnet</i>	Scansiona la sottorete locale tramite richieste ARP, identifica l'IP target 192.168.50.101
<i>pwd</i>	-	Print Working Directory, conferma la posizione dell'operatore in /home/kali/bruteforce
<i>ls</i>	-	Elenca i file nella directory, confermando la presenza di bruteforce.py, password.txt e username.txt

Fase 2: Architettura dello Script – Inizializzazione e Funzione di Caricamento

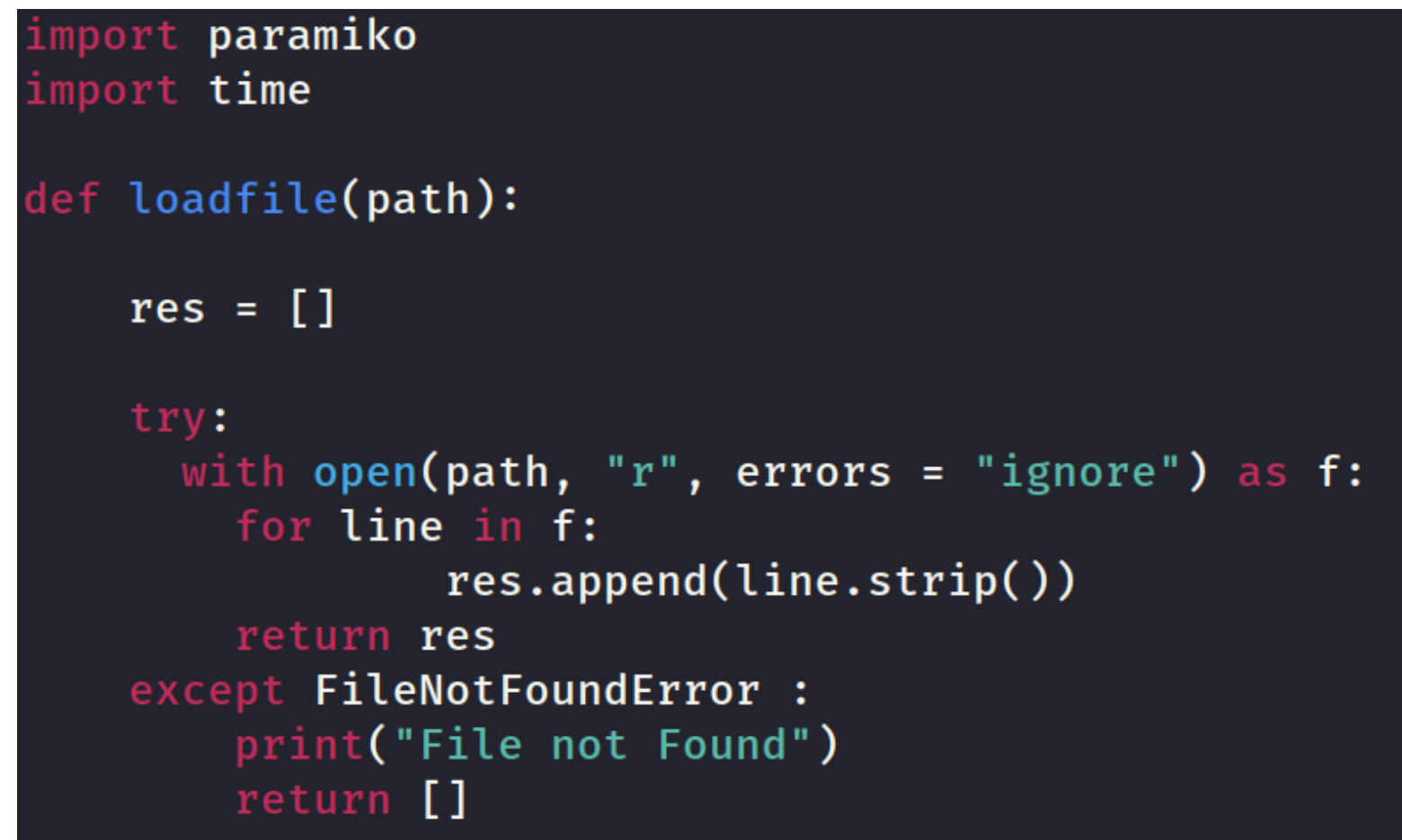


Fig. 2 — [Script iniziale che includono le librerie, la funzione personalizzata, funzioni e metodi built-in]

Prima di lanciare l'attacco, viene strutturato il **codice Python** per gestire l'importazione dei moduli necessari e la lettura dei dizionari di credenziali memorizzati su disco, facendo riferimento alla Figura 2, si osserva la prima porzione del codice sorgente, in cui vengono importate le librerie di sistema e viene definita la **funzione custom loadfile(path)**, progettata per leggere i file di testo riga per riga, pulire i caratteri invisibili (come i ritorni a capo) e popolare una lista in memoria gestendo eventuali errori di file mancante. Analizzando lo script possiamo individuare varie componenti principali del codice tra cui:

- **import paramiko**; importa la libreria esterna **Paramiko** che è il motore del programma: permette a Python di interagire con il **protocollo SSHv2**, creando client o server e gestendo l'autenticazione
- **import time**; importa il modulo standard di sistema per la gestione del tempo, verrà utilizzato successivamente per inserire una pausa temporale tra i vari tentativi di login
- **def loadfile(path)**; definisce una funzione personalizzata chiamata **loadfile** che accetta un unico argomento (**path**), ovvero il percorso del file di testo da leggere (la wordlist)
- **with open(path, "r", errors = "ignore") as f:** ; apre il file indicato da path in modalità sola lettura ("r")
 - L'argomento **errors = "ignore"** serve a ignorare eventuali caratteri non standard o problemi di codifica presenti nel dizionario
 - Il costrutto **with ()** garantisce che il file venga chiuso automaticamente e correttamente appena si esce dal blocco, anche in caso di errori

- **for line in f:** , avvia un ciclo che legge il file f riga per riga e ad ogni iterazione, il testo della riga corrente viene memorizzato nella variabile **line**
- **res.append(line.strip())** ; prende la stringa contenuta in line, applica il **metodo .strip()** per rimuovere gli spazi bianchi e soprattutto i caratteri invisibili di fine riga, e infine aggiunge (**.append()**) la parola pulita all'interno della lista **res**
- **return res** ; se la lettura del file si conclude con successo, la funzione termina restituendo la lista **res** popolata con tutte le credenziali
- **except FileNotFoundError :** ; intercetta l'errore specifico che si verifica se il percorso fornito a **open()** non corrisponde a nessun file esistente sul disco
- **return []** ; in caso di errore, la funzione restituisce una lista vuota [], questo previene il crash del programma principale quando tenterà di ciclare sulle credenziali

Nome Elemento	Libreria/Argomenti	Funzione Tecnica
<i>paramiko</i>	import paramiko	Modulo core utilizzato per implementare il protocollo client SSHv2 e gestire le richieste di autenticazione
<i>time</i>	import time	Modulo di sistema utilizzato per introdurre delay temporali durante l'esecuzione del ciclo di attacco
<i>loadfile(path)</i>	Definita nello script	Accetta un percorso file, lo apre in lettura ("r"), ignora gli errori di codifica, applica .strip() a ogni riga e restituisce la lista res
<i>open()</i>	Python Standard	Apre il file handler, il flag errors="ignore" evita il blocco del programma in presenza di caratteri non standard nei dizionari.
<i>.append()</i>	Oggetto Lista	Aggiunge la stringa della credenziale pulita in coda alla lista res
<i>.strip()</i>	Oggetto Stringa	Rimuove spaziature iniziali/finali e i caratteri di newline (\n, \r) da ogni riga letta

Fase 3: Logica di Attacco (Nested Loop) e Configurazione SSH

Fig. 3— [Ingresso del Programma, logica dell’attacco combinatorio]

```
if __name__=="__main__":

    #costante#
    SERVERURL = "192.168.50.101"
    USERNAME_PATH = "/home/kali/bruteforce/username.txt"
    PASSWORD_PATH = "/home/kali/bruteforce/password.txt"

    #Richiamo la funzione loadfile e carico nelle variabili i valori nei file#
    usernames=loadfile(USERNAME_PATH)
    passwords=loadfile(PASSWORD_PATH)

    print(usernames,passwords)

    for username in usernames:
        for password in passwords:
            try:
                #creazione client paramiko#
                client = paramiko.SSHClient()
                #Aggiungo al client la policy per gestire gli host sconosciuti#
                client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
                client.connect(SERVERURL , username=username , password=password)
                print("successo > ", username , password , "corretti")
                #trovati, fine programma
                exit(0);
            except paramiko.ssh_exception.AuthenticationException:
                print("username:",username," errato")
                print("password:",password,"errato")

            except paramiko.SSHException as e:
                print("SSHException", e)
            finally :
                client.close()
            # delay random che fa a fine di ogni esecuzione della coppia username, password
            #Questa riga mette in pausa l'esecuzione del programma per un intervallo di tempo
            time.sleep(1)
```

La logica di attacco si basa su parametri fissi (costanti) e su un doppio ciclo iterativo che mette alla prova ogni singola combinazione di utente e password, come mostrato nella Figura 3 il blocco principale (**__main__**) definisce l'IP del server e i percorsi dei dizionari, successivamente, richiama la **funzione loadfile** per popolare le liste in memoria ed avvia i **cicli for**, per ogni tentativo, viene istanziato un **client SSH Paramiko**, viene **configurata la policy** per ignorare il controllo interattivo delle chiavi host sconosciute, e viene tentata la connessione. In caso di fallimento della credenziale, l'eccezione viene intercettata per consentire il proseguimento del ciclo; tra una combinazione e l'altra è inserita una pausa di 1 secondo (**time.sleep(1)**). Analizzando la figura vediamo che contiene il punto di ingresso del programma (**Main**) e la logica vera e propria dell'attacco combinatorio:

- **if __name__ == "__main__":** ; è il costrutto standard di Python che indica il punto di partenza dell'esecuzione, in cui dice al programma: "Esegui il codice che segue solo se questo script viene lanciato direttamente, e non se viene importato come modulo in un altro script"
- **SERVERURL = "192.168.50.101" , USERNAME_PATH = "/home/kali/bruteforce/username.txt" , PASSWORD_PATH = "/home/kali/bruteforce/password.txt"** ; sono le costanti dedicate alle configurazioni fisse, in ordine definiscono: l’indirizzo IP della macchina target, il percorso assoluto sul file system contenente l’elenco degli usernames e il percorso assoluto sul file system contenente l’elenco delle password
- **usernames=loadfile(USERNAME_PATH) , passwords=loadfile(PASSWORD_PATH)** ; sono le chiamate della funzione, passa il percorso degli username e delle passwords e memorizza le liste risultanti nelle variabili **“usernames”** e **“passwords”**

- **for username in usernames: , for password in passwords: ;** inizio dei cicli (esterno ed interno), prende uno alla volta i nomi utente dalla **lista usernames**, il codice verrà eseguito per ogni nome utente, invece per l'utente attualmente selezionato nel ciclo esterno, il secondo ciclo proverà tutte le password presenti nella **lista passwords** una dopo l'altra, creando un attacco combinato
- **try:** ; apre un blocco di protezione per la gestione degli errori legati alla connessione di rete e ai fallimenti di login
- **client = paramiko.SSHClient()** ; iper-istanzia l'oggetto **SSHClient** della libreria **Paramiko** e lo assegna alla variabile **client**, questo oggetto espone i metodi necessari a connettersi
- **client.set_missing_host_key_policy(paramiko.AutoAddPolicy())** ; configura il client in modo che accetti e salvi automaticamente la chiave crittografica dell'host remoto (il server target) se questa non è presente nel file locale known_hosts, senza questa riga, lo script si bloccherebbe richiedendo un input interattivo all'utente
- **client.connect(SERVERURL , username=username , password=password)** ; tenta la connessione e l'**autenticazione SSH** verso il server target usando la coppia corrente di username e password, se le credenziali sono errate, questa riga interrompe il blocco bloccandosi immediatamente e lanciando un'eccezione, al contrario se sono corrette, il flusso prosegue alla riga successiva
- **except paramiko.ssh_exception.AuthenticationException:** ; cattura l'eccezione generata quando la password o l'utente inseriti sono errati, quando il server ha risposto "**Access Denied**".
- **except paramiko.SSHException as e:** ; cattura errori generali del protocollo SSH non legati alle credenziali (es. timeout di rete, sovraccarico del server, reset della connessione) e salva l'errore nella variabile **e**
- **finally :** ; un blocco speciale che viene sempre eseguito alla fine del costrutto **try/except**, indipendentemente dal fatto che l'autenticazione sia fallita o che sia avvenuto un errore di rete
- **client.close()** ; chiude il socket di rete e libera le risorse associate all'oggetto client, è fondamentale per **evitare di lasciare appese centinaia di connessioni TCP** aperte verso il server target, il che causerebbe un **Denial of Service (DoS)** o il blocco del test
- **time.sleep(1)** ; sospende l'esecuzione del programma per esattamente 1 secondo prima di passare alla successiva combinazione del **ciclo for**, questo delay evita di saturare di richieste la CPU e la banda della macchina target, riducendo parzialmente il rumore nei log

Nome Elemento	Valore Origine	Funzione Tecnica
<i>SERVERURL</i>	192.168.50.101	Stringa contenente l'indirizzo IP della macchina target
<i>USERNAME_PATH</i>	"/home/kali/..."	Percorso assoluto del dizionario utenti (username.txt)
<i>PASSWORD_PATH</i>	"/home/kali/..."	Percorso assoluto del dizionario password (password.txt)
<i>client</i>	paramiko.SSHClient()	Istanza del client SSH utilizzata per effettuare i tentativi di login
<i>SSHClient()</i>	paramiko	Inizializza l'oggetto per la sessione SSH
<i>set_missing_host_key_policy()</i>	paramiko.AutoAddPolicy()	Imposta il client in modalità di auto-accettazione delle chiavi host sconosciute, evitando blocchi di input interattivi
<i>connect()</i>	Paramiko Client	Invia la richiesta di connessione SSH impostando hostname, username e password
<i>exit(0)</i>	Python Standard	Interrompe immediatamente lo script con codice di successo (0) al primo login valido
<i>close()</i>	Paramiko Client	Invocato nel blocco finally:, assicura la chiusura del socket SSH dopo ogni tentativo (sia esso fallito o andato in errore)
<i>sleep(1)</i>	Modulo time	Sospende l'esecuzione per 1 secondo alla fine di ogni tentativo, agendo come controllo della velocità di attacco

Fase 4: Esecuzione del Test e Monitoraggio Output

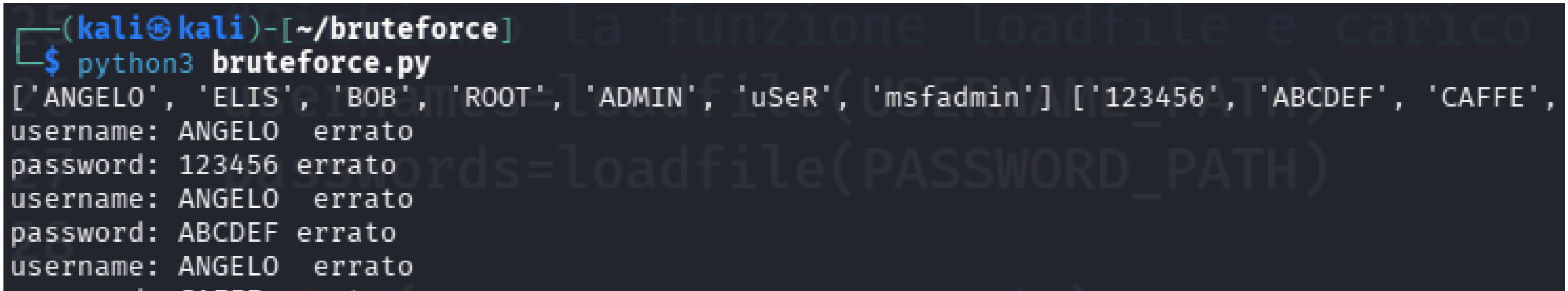


Fig. 4 — [Avvio del Programma, con i dizionari degli username e delle password caricati a memoria]

Una volta validato il codice, l'operatore avvia lo script dal terminale Linux, **monitorando visivamente il caricamento dei vettori d'attacco in memoria e l'inizio della sequenza di brute force**, prendendo in esame la Figura 4 si osserva il lancio dello script tramite l'interprete Python 3 e immediatamente prima di iniziare l'attacco lo script stampa a video il contenuto dei due dizionari caricati in memoria sotto forma di liste Python, subito dopo, che iniziano i tentativi sequenziali partendo dall'utente ANGELO combinato con le diverse password presenti nel dizionario

Nome Elemento	Tipo	Funzione Tecnica
<i>python3 bruteforce.py</i>	Comando Linux (CLI)	Esegue l'interprete Python 3 eseguendo lo script di attacco nella directory corrente
<i>usernames</i>	Variabile (Lista)	Array contenente la lista completa dei nomi utente letti dal file di testo e stampati a schermo
<i>passwords</i>	Variabile (Lista)	Array contenente l'elenco completo delle password caricate in memoria e stampate a schermo

Fase 5: Successo dell'Attacco ed Estrazione delle Credenziali

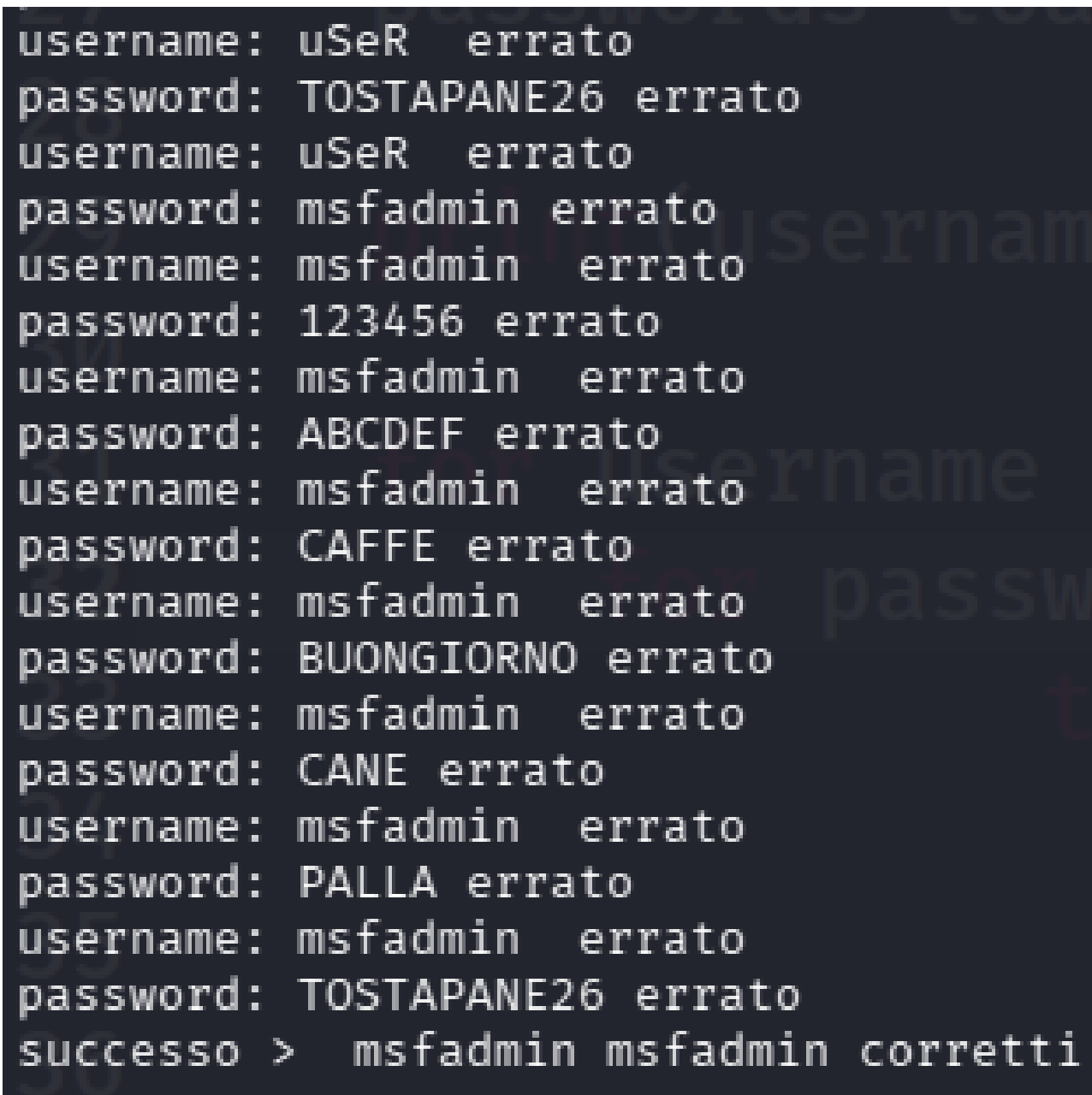


Fig. 5 — [Programma Terminato con Successo, conferma della compomissione dell'account]

I cicli iterativi proseguono scartando sistematicamente le credenziali errate fino a quando i parametri inviati alla funzione **.connect() non soddisfano i requisiti di autenticazione del server SSH target**. Come documentato dettagliatamente nella Figura 5 lo schermo mostra il log continuo dei tentativi falliti (contrassegnati dalla stringa errato), come l'utente uSeR o i tentativi mirati sull'utente msfadmin con password errate (es. 123456, ABCDEF, CAFFE, BUONGIORNO, CANE, PALLA, TOSTAPANE26), l'attacco si interrompe bruscamente e **con successo non appena viene testata la combinazione valida**:

- Username identificato: **msfadmin**
- Password identificata: **msfadmin**

A quel punto, il programma esegue l'istruzione di stampa di successo (**successo > msfadmin msfadmin corretti**) ed esce dall'esecuzione (**exit(0)**), confermando la compromissione dell'account

Analisi della Gestione delle Eccezioni

La continuità operativa dello script durante le risposte negative fornite dal server target è garantita dalla gestione mirata degli errori strutturata nel **blocco try-except** (analizzato nella figura 3):

- paramiko.ssh_exception.AuthenticationException**: Questa eccezione viene sollevata **ogni volta che il server SSH rifiuta la combinazione utente/password**, il blocco except cattura l'errore, stampa a video lo stato errato (come si vede nella Figura 5) e permette al ciclo for di passare alla password successiva senza mandare in crash lo script.
- paramiko.SSHException**: Gestisce errori di livello inferiore legati al protocollo SSH, lo script ne cattura l'istanza come **e** e la stampa a terminale.
- FileNotFoundError**: Se uno dei dizionari non viene trovato nel percorso indicato, **stampa File not Found e restituisce una lista vuota []**, impedendo l'interruzione anomala dell'intero script in fase di avvio(Figura 5)

Contromisure e Raccomandazioni Tecniche

L'esito positivo del test condotto nel laboratorio evidenzia la necessità di implementare le seguenti mitigazioni sul sistema target (192.168.50.101):

- **Rimozione o Modifica delle Credenziali di Default:** L'account `msfadmin:msfadmin` rappresenta una configurazione predefinita **altamente insicura**, la password deve essere modificata immediatamente adottando criteri di complessità elevati, oppure l'account deve essere disabilitato
- **Sistemi di Account Lockout e Rate Limiting:** Il target non ha interrotto la sessione né bloccato l'IP attaccante nonostante i numerosi tentativi falliti consecutivi, è **indispensabile installare utility per monitorare i log di SSH e bloccare temporaneamente tramite firewall l'indirizzo IP dell'attaccante dopo un numero ridotto di fallimenti** (es. 3 o 5 tentativi)
- **Autenticazione tramite Chiave Asimmetrica:** Si raccomanda di **disabilitare completamente l'autenticazione basata su password all'interno della configurazione del demone SSH**, forzando esclusivamente l'uso di chiavi crittografiche pubbliche/private.

Allegato

[Clicca qui per bruteforce.py](#)